

Immersive Software Engineering

EPIC: Refactoring the iDaVIE Codebase for Maintainability

Assignment Specification

Duration: 15 working days

Mon 18 May 2026 (09:00) - Fri 5 June 2026 (16:00)

1. Module Information and Context

Item	Detail
Assessment title	Refactoring proposal for the iDaVIE codebase to improve maintainability against ISO/IEC 25010:2023
Cohort	55 first-year undergraduate Immersive Software Engineering students.
Team structure	Two competing teams: Team Alpha (28 students) and Team Beta (27 students). Asymmetric by design — see Section 5.
Duration	15 working days: Mon 18 May 2026 09:00 - Fri 5 June 2026 16:00. Daily 09:00-17:00 (Fri 5 June ends 16:00). Breaks: 10:00-10:30, 12:00-13:00, 14:00-14:30. Documented exceptions apply on the pitch day and on the two final interview days.
Process model	Three-sprint Scrum, scaled to a 28-person team using a Scrum-of-Scrums pattern plus Architecture and Quality Guilds. CI/CD workflow from Day 2.
Codebase under study	iDaVIE: Immersive Data Visualization Interactive Explorer. C# + C/C++ native plug-ins on Unity. https://github.com/idia-astro/iDaVIE/
Mode of work	Design-only refactoring proposal. No upstream code is changed; each team demonstrates with worked examples how the code would be changed if the proposal were adopted. Software construction competency is exercised vicariously through design-level worked refactoring examples.
Final outcome	Each team pitches its solution to the iDaVIE maintainer panel on Thu 4 June 2026. The winning proposal forms the basis for a future structured refactoring effort.

1.1 The iDaVIE Codebase

iDaVIE is an open-source VR application for immersive visualisation of 3D astronomical data cubes (FITS) built on the Unity game engine and the SteamVR interaction stack. The current production architecture is broadly: a single Unity scene driven by MonoBehaviour C# scripts; performance-critical I/O implemented in C/C++ native DLLs (FitsReader, DataAnalysis, AstTool); a Rendering Layer (VolumeDataSetRenderer + shaders) doing GPU ray-marching; a Feature system for regions of interest; a VR Interaction System with two state machines (LocomotionState, InteractionState); and a Desktop GUI (CanvassDesktop) alongside the VR view. Known maintainability pressures: tight coupling between MonoBehaviours and scene state; heavy reliance on singletons; long monolithic classes; mixed concerns in the rendering and interaction layers; Unity lifecycle code intermixed with domain logic; thin abstraction over

native plug-ins; limited automated test coverage. These pressures are amplified by an imminent platform migration from Unity 5-era patterns to Unity 6.

1.2 Strategic Drivers

1. **Maintainability:** ISO/IEC 25010:2023 maintainability: modularity, analysability, modifiability and testability, plus reusability as a supporting concern.
2. **Platform migration:** a technology-stack shift from Unity 5 to Unity 6, i.e. new Input system, scriptable render pipelines, package-based architecture, modern C#. The refactored architecture must make this migration routine.

2. Learning Outcomes

On successful completion of this assessment, every student will be able to:

LO	Statement	Competency area
L01	Benchmark the maintainability of a real-world C#/C++ codebase using industry-standard static analysis tools and the Chidamber and Kemerer (CK) object-oriented metrics suite, and interpret the resulting evidence.	Analysability
L02	Elicit, document and prioritise non-functional requirements in particular the ISO/IEC 25010 maintainability sub-characteristics and translate them into testable architectural drivers.	Requirements engineering
L03	Design a client–server software architecture using a micro-kernel pattern for the backend with a layered architecture inside the kernel and a C/C++ plug-in mechanism, and justify the decision against alternatives.	Software architecture
L04	Apply SOLID and GRASP principles at the class and component level to produce loosely coupled, highly cohesive units of code with clear separation of concerns.	Software design
L05	Specify a refactoring of a legacy Unity 5 codebase to a target Unity 6 stack with UML/SysML diagrams, dependency graphs, and worked design-level refactoring examples evidencing the construction competency.	Software construction (vicarious)
L06	Design a testability strategy including branch/line and condition coverage targets, dependency isolation, mocking strategies for Unity-bound code, and CI-driven quality gates.	Software testing
L07	Operate as a sub-team within a 28-person team using Scrum-of-Scrums, with daily stand-ups, Kanban tracking, CI/CD, and demonstrated cross-sub-team integration.	Project management
L08	Use AI/GenAI tools effectively and responsibly across the software lifecycle, explain and defend AI-assisted output as authors of record, and identify where AI tools failed.	Professional practice
L09	Communicate and defend an architecture and refactoring proposal to a real client, the iDaVIE maintainer team, in a competitive pitch setting, including trade-off analysis.	Communication

3. SWEBOK Knowledge Areas

This assessment maps to the following Software Engineering Body of Knowledge (SWEBOK) Knowledge Areas (KAs). Each is exercised explicitly by at least one sub-team work package (Section 6).

SWEBOK KA	How exercised in this assessment
Software Requirements	Every sub-team elicits and documents non-functional requirements traceable to the four ISO/IEC 25010 sub-characteristics; architecturally significant requirements are owned by Sub-team 1.
Software Architecture	Client–server + micro-kernel + layered + plug-in target style. C4 model documentation. Architecture Decision Records (ADRs).
Software Design	Class, component and sequence diagrams; SOLID and GRASP audit; design trade-off analysis.
Software Construction	Exercised through worked refactoring examples (UML before/after, dependency graphs, CK metrics deltas). No production code is changed.
Software Testing	Testability strategy per sub-team; branch/line and condition coverage targets; mocking and dependency isolation; CI-enforced quality gates.
Software Maintenance	The whole assessment is a maintenance exercise: corrective and preventive refactoring evidence against an ISO/IEC 25010 baseline.
Software Configuration Management	Git on GitHub fork; ADRs as configuration items; CI pipeline definitions; metric dashboards versioned.
Software Engineering Management	Scrum-of-Scrums with three-layer roles; Kanban tracking; sprint cadence; integration risk register.
Software Engineering Process	Daily stand-ups, sprint planning, sprint review, retrospective. CI/CD pipeline operated from Day 2.
Software Quality	ISO/IEC 25010 maintainability sub-characteristics; CK metrics; quality gates; baseline vs. projected post-refactor evidence.
Software Engineering Professional Practice	AI/GenAI tool use, authorship, peer-rating, individual reflection, defensible decisions.

4. The Target Architecture

Both teams target the same high-level architectural style. Teams compete on how well they realise that style, not on inventing a different one.

4.1 Style Summary

- **Client–server** at the system level. The Unity 6 VR client handles rendering, input and presentation. The server hosts data, domain logic, plug-in execution and long-running computations.
- **Micro-kernel** for the server. A small, stable kernel exposes a versioned plug-in contract; data formats, domain operations and coordinate systems are realised as C/C++ plug-ins.
- **Layered architecture inside the kernel:** Domain → Application → Infrastructure → Plug-in host, strict downward dependency only.
- **Anti-corruption layer** around Unity 6 APIs so domain logic in the client is testable outside Unity, and so the Unity 5 to Unity 6 migration is contained.

4.2 Mandatory Architectural Constraints

1. No unit of code (class or component) may violate SOLID or GRASP. Violations must be flagged and refactored, or justified as a documented trade-off.
2. No circular dependencies between top-level components. Cyclic package dependencies are a fail condition.
3. Domain code, i.e. rendering math, FITS parsing, feature analysis etc. must not transitively depend on UnityEngine or SteamVR types.
4. Every public API boundary between layers/components must be expressed as an interface and covered by at least one test double.
5. The plug-in contract for C/C++ extensions must be versioned, i.e. semantic versioning, and ABI-stable within a major version.

5. Team and Sub-team Structure

5.1 Two Competing Teams (Asymmetric)

The cohort of 55 students are split into Team Alpha (28 students) and Team Beta (27 students). The two teams work independently and competitively. The asymmetric one-student difference is by design and is acknowledged at the pitch.

5.2 Sub-team Structure

Team	Sub-team count	Composition	Total
Team Alpha	7	7 sub-teams of 4 students	28 students
Team Beta	7	6 sub-teams of 4 + 1 sub-team of 3 (Persistence & Workspace State)	27 students

Both teams cover the same 7 canonical work packages described in Section 6. The single 3-person sub-team is in Team Beta's Persistence & Workspace State work package and operates under reduced-scope rules; see Section 6.7 and Section 10.4.

5.3 Capacity-Asymmetry Policy

Team Alpha has one additional student and a full-size Persistence sub-team; Team Beta's Persistence sub-team has 3 students with reduced scope. To ensure fair grading despite this asymmetry, the following policies apply throughout this specification:

- Both teams are graded on the depth, rigour and quality of their proposal, not on the raw count of worked examples. Volume differences attributable to team capacity do not affect team marks.
- Team Beta's Persistence sub-team is graded on quality at its adjusted scope, with full marks accessible at half scope. See Section 10.4.
- The Team PO and Team SM eligibility pool differs between teams (Section 10): Team Alpha draws from all 28 students; Team Beta draws from 24 students in the six four-person sub-teams. Elevation availability does not influence grading.

5.4 Sub-team Responsibilities and Independence

- Each sub-team owns a distinct work package representing a major behavioural element of the iDaVIE codebase (Section 6).
- Each sub-team works independently of the overall team. Independence here means self-sufficient on its allocated scope and not blocked waiting on others; it does not mean isolation from coordination.

- Each sub-team integrates its sub-solution into the overall team solution via explicit interface contracts.
- No single sub-team is responsible for coordinating quality, CI/CD, metrics or benchmarking. These are owned collectively by the Quality Guild (Section 10).

5.5 Sub-team Allocation

The 14 sub-teams across both competing teams are allocated to work packages as follows. Allocation is the assessment-authoritative mapping; both teams cover the same 7 canonical work packages. The Team Beta Persistence sub-team is the only 3-person sub-team.

Team #	Sub-team name	Competing team	Work package
1	Apaties I	Team Alpha	Architecture and Micro-kernel Core
12	Babelaas	Team Alpha	Data I/O and FITS/WCS Plug-ins
13	Cache Me If You Can	Team Alpha	Rendering Engine
4	Koffiewinkel	Team Alpha	Interaction System (VR, voice)
2	Apaties II	Team Alpha	Feature System and Domain Model
5	Die Boks	Team Alpha	Desktop GUI and Client Shell
14	Sewe en sestig	Team Alpha	Persistence and Workspace State (4-person)
6	Terminal Ses	Team Beta	Architecture and Micro-kernel Core
10	Apaties IV	Team Beta	Data I/O and FITS/WCS Plug-ins
3	kameel-case	Team Beta	Rendering Engine
9	Drie Bums en 'n Al-Broer	Team Beta	Interaction System (VR, voice)
7	kebab_case	Team Beta	Feature System and Domain Model
8	Apaties III	Team Beta	Desktop GUI and Client Shell
11	Apaties V	Team Beta	Persistence and Workspace State (3-person)

6. Sub-team Work Packages

Each of the seven canonical work packages exists in both Team Alpha and Team Beta. Every work package demonstrates evidence of all six SWEBOK V4 competencies: requirements engineering, software architecture, software design, software construction (vicariously through worked refactoring examples), software testing, and software engineering management.

6.1 Sub-team 1: Architecture and Micro-kernel Core

Behavioural element owned	The kernel boundary, the plug-in ABI, the layer dependency policy, and the integration spine of the whole team.
Strategic role	Custodian of the architecture, the plug-in contract, and the dependency policy. Integration point for the whole team.

Requirements Engineering

- Translate ISO/IEC 25010 sub-characteristics into architecturally significant requirements (ASRs).
- Define kernel NFRs: load time, ABI stability, plug-in failure isolation, hot-reload, observability.
- Capture stakeholder expectations from the iDaVIE roadmap (subcube loading, HDU selection, particle datasets, multiplayer).

Software Architecture

- Top-level component diagram (UML) and SysML BDD of the client–server split.
- Layered architecture inside the kernel (Domain / Application / Infrastructure / Plug-in Host) with strict downward dependency rule.
- C ABI for plug-in contract: symbol versioning, error model, threading model, memory ownership, ABI compatibility policy.
- Architecture Decision Records (ADRs) and the architecture violation policy enforced in CI.

Software Design

- Dependency Inversion at every layer boundary.
- Plug-in registry as a Service Locator at the kernel boundary only; constructor injection inside the kernel (GRASP: Indirection, Protected Variations).

Software Construction: Worked refactoring examples

- Annotated before/after UML class diagram for one current god-like class (e.g., VolumeDataSetRenderer).
- Sequence diagram: 'user loads FITS cube, requests subcube, exports mask'.

Software Testing

- Architecture-level fitness functions (ArchUnit-style); NDepend / DV8 layer-violation rules in CI.
- Contract test suite for any C/C++ plug-in to be considered conformant.

Software Engineering Management

- Architecture Kanban, ADR backlog, integration risk register.
- Chair the Architecture Guild: daily cross-sub-team stand-up at 09:00 / 08:55 on interview days.

Sub-team Deliverables

- Architecture overview (10–15 pages) with C4 model.
- Plug-in ABI specification (versioned).
- ADR log (minimum 8 ADRs).
- Layer-violation CI check definitions.

6.2 Sub-team 2: Data I/O and FITS/WCS Plug-ins

Behavioural element owned	The C/C++ data plumbing — FITS reading and writing, VOTable, WCS coordinate transformations, downsampling, statistics — re-cast as kernel plug-ins.
Strategic role	Owner of the data plumbing. Closest sub-team to the legacy native DLLs.

Requirements Engineering

- Document current FitsReader, AstTool and DataAnalysis behaviour from published descriptions.
- Catalogue upcoming requirements: subcube loading, HDU selection beyond HDU 0, MUSE/NIRSpec/MIRI compatibility, streaming.

Software Architecture

- FitsReader, AstTool, DataAnalysis recast as three independently-loadable plug-ins behind the kernel ABI.
- Boundary between data-format plug-ins (FITS, VOTable, TIFF) and data-operation plug-ins (statistics, downsampling, source-finding).

Software Design

- Single Responsibility split of FitsReader (parsing, indexing, caching, axis selection).
- Strategy pattern for downsampling; Adapter for legacy CFITSIO/AstLib bindings.
- GRASP Information Expert for statistical calculations co-located with their data structures.

Software Construction: Worked refactoring examples

- Before/after of one FitsReader method, including ownership of unmanaged memory.
- WCS coordinate transformations as pure function plug-in with no Unity dependency.

Software Testing

- Reference FITS corpus (synthetic + public) for plug-in conformance testing.
- Property-based tests for round-trip FITS read/write and WCS transformations.
- Isolation strategy so plug-in tests run without Unity.

Software Engineering Management

- Sub-team Kanban with explicit dependencies on Sub-team 1 (ABI) and Sub-team 3 (texture format).
- Daily sync with Sub-team 3 for texture interop.

Sub-team Deliverables

- Plug-in design document for each of the three data plug-ins.
- C ABI binding header samples.
- Conformance test plan.

6.3 Sub-team 3: Rendering Engine

Behavioural element owned	Volume rendering - ray-marching shaders, 3D texture management, foveated rendering, colour mapping - and its migration to Unity 6 SRP.
Strategic role	Owner of the volume rendering layer in the client and its Unity 6 scriptable render pipeline migration.

Requirements Engineering

- Current invariants: 90 fps floor, 4 GB Unity texture limit, 368 MB default cube budget, blocky filtering, foveated rendering.
- Future: iso-contours/surfaces, multi-cube/time-series.

Software Architecture

- Rendering as one client component with inward dependency on the volume domain model and outward on URP/HDRP.
- Render-pipeline abstraction so the rendering core does not transitively depend on URP/HDRP-specific types.

Software Design

- Open–Closed for mask modes (Apply/Inverse/Isolate) as Strategy behind one IMaskMode interface.
- Split VolumeDataSetRenderer into VolumeMaterialBinder, VolumeTextureManager, VolumeCameraDriver, FoveatedSamplingPolicy.

Software Construction: Worked refactoring examples

- Before/after class structure of VolumeDataSetRenderer with dependency graph and CK metrics deltas (WMC, DIT, NOC, CBO, RFC, LCOM).
- Sequence diagram for one rendered frame in the new architecture.

Software Testing

- Play-mode tests under Unity Test Framework for renderer behaviour; edit-mode tests for non-Unity parts.
- Golden-image regression suite across mask modes and colour maps.

Software Engineering Management

- Sub-team Kanban with dependencies on Sub-team 2 (texture data) and Sub-team 4 (camera/locomotion).
- Performance budget review at each sprint boundary.

Sub-team Deliverables

- Rendering layer design document.
- Shader/asset organisation policy for Unity 6.
- Before/after metrics worksheet for the renderer.

6.4 Sub-team 4: Interaction System (VR, voice, controllers)

Behavioural element owned	LocomotionState and InteractionState machines, voice commands, quick-menu, paint-menu — re-platformed onto Unity 6 Input System.
Strategic role	Owner of the user-facing interaction layer.

Requirements Engineering

- Every current state transition documented: Idle, Moving, ScalingRotating, Parameter Editing, Creating Selection, Editing Region, Source Editing, Painting Stroke.
- Accessibility and i18n for voice commands (push-to-talk, sensitivity, locale).

Software Architecture

- Interaction layer as client component with no SteamVR-specific direct dependency; an IInputProvider abstraction.
- State machines as reusable, Unity-independent classes with a thin Unity adapter.

Software Design

- State pattern formalised for both state machines (currently implicit).
- Command pattern for voice commands and quick-menu actions — reified, replayable, testable.
- Interface Segregation: IPointer, IGripInput, IVoiceInput, IHaptics.

Software Construction: Worked refactoring examples

- LocomotionState in target shape with formal State classes; UML state machine diagram + C# skeleton.
- Windows KeywordRecognizer dependency replaced with IVoiceRecogniser interface and one concrete adapter.

Software Testing

- Pure C# unit tests for state machines using mock IInputProvider and IVoiceRecogniser.
- Scenario tests for VR flows: select region → crop → paint mask → save.

Software Engineering Management

- Sub-team Kanban with dependencies on Sub-team 3 (camera state) and Sub-team 5 (feature creation).
- Weekly UX review with at least one student from outside the sub-team.

Sub-team Deliverables

- Interaction layer design document.
- State machine diagrams (UML).
- Input abstraction header (C# interfaces).

6.5 Sub-team 5: Feature System and Domain Model

Behavioural element owned	Masked, Imported and User-defined features; FeatureSetManager; source-list statistics; moment maps; spectral profiles; VOTable export.
Strategic role	Owner of the Feature domain.

Requirements Engineering

- Current behaviour of the three Feature flavours and their realtime statistics (voxel count, total/peak flux, flux-weighted centroid, W20).
- Future: iso-contours/surfaces, particle datasets, Virtual Observatory integration.

Software Architecture

- Feature promoted to a first-class domain aggregate in the kernel domain layer, independent of any Unity type.
- FeatureSetManager split into FeatureCatalog (persistence + identity), FeatureSetService (use-case orchestration), FeatureVisualiser (Unity-side adapter).

Software Design

- GRASP Information Expert for feature-derived statistics on the Feature aggregate.
- Liskov: Masked, Imported, User-defined features substitutable behind one IFeature interface.

Software Construction: Worked refactoring examples

- Moment-map generation: from Unity script directly calling DataAnalysis DLL to a use case on the server returning a result.
- VOTable export as a FeatureCatalog responsibility with a clear export plug-in seam.

Software Testing

- Property-based tests for feature statistics (centroid inside bbox; flux non-negative; $W20 \leq W50$).
- Scenario test: mask → masked features → edited feature → exported VOTable.

Software Engineering Management

- Sub-team Kanban with dependencies on Sub-team 2 (data) and Sub-team 4 (creation flow).

Sub-team Deliverables

- Feature domain design document.
- Feature aggregate UML class diagram + invariants list.
- Worked statistics test specification.

6.6 Sub-team 6: Desktop GUI and Client Shell

Behavioural element owned	CanvassDesktop, file/mask loaders, parameter panels, debug consoles, and the client-side composition root that wires everything to the server.
Strategic role	Owner of the client shell.

Requirements engineering

- Current desktop GUI behaviour (File, Render, Stats, Sources, Debug tabs); direct file I/O that belongs server-side.
- Long-term: Python console, workspace/state saving.

Software architecture

- MVVM-style split: View (Unity 6 UI Toolkit), ViewModel (pure C#), Service Gateway (talks to the server).
- Client-server transport contract specified: JSON-RPC over named pipes for local mode; gRPC for future remote streaming.

Software Design

- Single Responsibility on CanvassDesktop: menu structure, panel state, file dialogs and configuration separated.
- Composable panels with explicit contracts (not a single God-canvas).

Software Construction: Worked refactoring examples

- File tab from direct native-plugin call to ViewModel command via service gateway.
- Debug tab as Observer of a structured logging stream.

Software Testing

- ViewModel unit tests (no Unity required).
- UI-Toolkit page-object pattern for required integration tests.

Software Engineering Management

- Sub-team Kanban with dependencies on Sub-team 1 (service gateway) and Sub-team 4 (VR-side menus).

Sub-team Deliverables

- Desktop client architecture document.
- MVVM binding policy.
- Worked refactoring of File and Debug tabs.

6.7 Sub-team 7: Persistence and Workspace State

Behavioural element owned: the lifecycle of a user's iDaVIE session, i.e. loaded cubes, mask edits, paint strokes, defined features, view parameters, selection boxes, render settings etc. from capture to durable storage to restore, including crash recovery and forward-compatible versioning.

6.7.1 Team Alpha: Full-scope (4-person sub-team)

Standard scope, identical structure to the other six work packages:

- Two worked refactoring examples with CK metrics deltas.
- Design document 5–10 pages.
- Full four-role rotation (SM, Tech Lead, PO Liaison, Quality Champion) across three sprints.
- Standard module-staff support.

6.7.2 Team Beta: Reduced scope (3-person sub-team)

Adjusted scope reflecting 75 % capacity:

- One worked refactoring example with CK metrics delta.
- Design document 4–7 pages.

- Modified role rotation: SM and QC bundled (one student per sprint carries both); TL and POL as single roles. Each student does the SM+QC double once and each single role once across the three sprints.
- Bus-factor risk note: the 3-person sub-team operates with a bus factor of 1. If any member is absent for more than half a day, the fall-back applies the 2-person rules: SM+QC paired with one student; TL+POL paired with the other.

6.7.3 Shared Requirements, Design and Testing (Both Sub-Teams)

- Define what a workspace IS: domain model, aggregates, invariants.
- Persistence boundary: format, schema, versioning, migrations.
- Lifecycle: autosave cadence, transactional semantics, conflict resolution.
- State contracts with every other sub-team: each declares what its state looks like.
- Recovery path: partial state, corrupted state, version mismatch.
- Testability: pure-C# persistence layer with no Unity dependency; deterministic round-trip tests; property-based tests for migrations.

7. Maintainability Metrics (ISO/IEC 25010:2023)

Each sub-team produces two metric snapshots for its allocated code: a Day 2 baseline and a Day 13 projected snapshot demonstrating what the proposed refactoring would deliver. The projected snapshot must be supported by worked refactoring examples; speculative numbers without evidence are not accepted. No single sub-team coordinates metrics; the Quality Guild owns the team-level dashboard and consolidated reports.

7.1 Chidamber and Kemerer (CK) suite (Mandatory)

CK metric	Meaning	Acceptable range
WMC	Weighted Methods per Class.	≤ 20 domain; ≤ 40 adapters.
DIT	Depth of Inheritance Tree.	≤ 4 .
NOC	Number of Children.	≤ 5 .
CBO	Coupling Between Object classes.	≤ 14 domain; ≤ 25 orchestrators. Cycles forbidden.
RFC	Response For a Class.	≤ 50 .
LCOM	Lack of Cohesion in Methods (Henderson-Sellers variant).	≤ 0.5 .

7.2 Metric families by ISO/IEC 25010 sub-characteristic

Modularity

- Coupling (CBO, fan-in, fan-out, afferent/efferent at package level).
- Cohesion (LCOM, conceptual cohesion in CodeScene).
- Dependency cycles at namespace and assembly level (must be 0).
- Instability $I = Ce / (Ca + Ce)$ per Stable Dependencies Principle.
- Package/component dependencies as DSM (DV8 or NDepend).
- Architecture violations against the layer rule (must be 0 in the projection).

Analysability

- Cyclomatic complexity per method, per file.
- Cognitive complexity (SonarQube definition).
- LOC, method/class size, nesting depth.
- Duplication (% duplicated lines, 30-line block threshold).
- Code smells (severity-weighted).
- Documentation / comment density on public APIs.

Modifiability

- Change impact (files touched per representative change scenario).
- Propagation cost (DV8).
- Technical debt (SonarQube; CodeScene).
- Maintainability Index.
- Churn / change frequency over last 24 months of public history.
- Hotspots (CodeScene churn × complexity intersection).

Testability

- Cyclomatic complexity (lower = easier to cover).
- Branch and line coverage (≥ 70 % domain; ≥ 50 % overall; Unity-bound code tracked but not in strict target).
- Condition / decision coverage on critical paths.
- Dependency isolation index (proportion of classes with interface dependencies).
- Mocking difficulty (count of static / Unity API calls per class).
- Interface size (ISP target ≤ 7 public members).
- Fan-out and test-affecting code smells.

7.3 Mandated Tools

Operational by end of Day 2. Owned collectively by the Quality Guild:

- **SonarQube Cloud:** code smells, complexity, duplication, maintainability rating, technical debt, coverage.
- **Understand;** CK suite, structural metrics, dependency browsing.
- **NDepend:** CQLinq rules for architecture violations, instability, propagation cost, layer enforcement.
- **CodeScene:** hotspots, churn, knowledge map, code health, change coupling.
- **DV8:** Dependency Structure Matrix, propagation cost, architectural anti-patterns.

8. Schedule and Sprint Plan

15 working days organised as three sprints: a full 5-day Sprint 1, a full 5-day Sprint 2, and a compressed 'Finalise & Defend' Sprint 3 of approximately 2 effective working days plus delivery days. Daily 09:00–17:00 with breaks at 10:00–10:30, 12:00–13:00 and 14:00–14:30 except where noted. Friday 5 June ends at 16:00. The assessment opens with a 2-hour iDaVIE team presentation and overview on Day 1 morning (09:00–11:00).

8.1 Sprint 1: Understand and benchmark (Days 1–5, 18–22 May)

Day	Date	Focus
Day 1	Mon 18 May	Full-day opening. 09:00–11:00 iDaVIE team presentation and overview. 11:00–12:00 sub-team kick-off meetings). 12:00–13:00 lunch. 13:00–14:00 tooling and repository access provisioned, CI/CD scaffolding begins. 14:00–14:30 break. 14:30–17:00 initial codebase exploration; each sub-team reads its allocated work package brief. No Sprint 1 planning today.
Day 2	Tue 19 May	Sprint 1 planning (morning). Baseline benchmark by every sub-team for its allocated code. CI skeleton green by end of day.
Day 3	Wed 20 May	Concern mapping; sub-teams own initial scope; first ADR drafts (Sub-team 1).
Day 4	Thu 21 May	Requirements engineering deliverables drafted. First interface contracts proposed. Staff check-in 1/3 for Team Beta Persistence.
Day 5	Fri 22 May	Sprint review (15 min per sub-team) + retrospective. Sprint 2 plan committed.

8.2 Sprint 2: Design and worked refactoring (Days 6–10, 25–29 May)

Day	Date	Focus
Day 6	Mon 25 May	Component diagrams and SysML BDDs frozen.
Day 7	Tue 26 May	Worked refactoring examples 1/2 per sub-team drafted with before/after CK metrics.

Day 8	Wed 27 May	Cross-sub-team integration review.
Day 9	Thu 28 May	Worked examples 2/2 complete. SOLID/GRASP audit per example. Sprint 2 exit criterion: Architecture Guild signs off all 6 state contracts delivered to Persistence (both teams; mandatory for Team Beta). Staff check-in 2/3 for Team Beta Persistence.
Day 10	Fri 29 May	Sprint review + retrospective. Mid-assessment iDaVIE-team visit (30 min per team).

8.3 Sprint 3: Finalise and Defend (Days 11–13, 1–3 June)

Sprint 3 is compressed: only Days 11 and 12 are full working days; Day 13 (Wed 3 June) is mixed because interviews begin that morning. There is no formal sprint planning ceremony for Sprint 3; the team enters the sprint with deliverables substantially complete.

Day	Date	Focus
Day 11	Mon 1 Jun	Address mid-assessment feedback; trade-off analysis written up.
Day 12	Tue 2 Jun	Final sprint work day. All worked examples final; team-level Integration and Metrics Report (Quality Guild) drafted. 16:00–17:00: Sprint 3 retrospective within each sub-team only (no team-wide retro for Sprint 3). Artefact freeze: 17:00.
Day 13	Wed 3 Jun	Interviews begin 09:00. Six 60-min interviews single-track (see Section 8.4). Non-interviewed sub-team members finalise packaging.

8.4 Pitch and Interview Schedule (Days 13–15, 3–5 June)

Single-track interviews run consecutively over three days, evenly distributed (6 / 4 / 4), excluding the Thu 4 June pitch window. Defence-relevant interview time is capped at 15 min per student regardless of sub-team size; residual time within the 60-min slot is for sub-team-level discussion. On interview days, the standard 10:00–10:30 and 14:00–14:30 breaks are absorbed into between-interview transitions; only lunch is protected (and shifted to 13:00–14:00 on Thu and Fri).

Wednesday 3 June (09:00–16:00): 6 interviews

Time	Activity
09:00–10:00	Interview 1
10:00–11:00	Interview 2
11:00–12:00	Interview 3
12:00–13:00	Lunch
13:00–14:00	Interview 4
14:00–15:00	Interview 5
15:00–16:00	Interview 6

Thursday 4 June (09:00–11:00, 11:00–13:00 PITCH, 14:00–16:00): 4 interviews and 2 pitches

Time	Activity
09:00–10:00	Interview 7
10:00–11:00	Interview 8
11:00–12:00	Team Alpha Pitch (40 min presentation + 20 min Q&A)
12:00–13:00	Team Beta Pitch (40 min presentation + 20 min Q&A)
13:00–14:00	Lunch
14:00–15:00	Interview 9
15:00–16:00	Interview 10

Friday 5 June (09:00–13:00, 14:00–16:00): 4 interviews + submission

Time	Activity
09:00–10:00	Interview 11
10:00–11:00	Interview 12
11:00–12:00	Interview 13
12:00–13:00	Interview 14
13:00–14:00	Lunch
14:00–16:00	Brightspace submission window. Packaging only, no new content permitted. Artefact set frozen at Thu 4 June 11:00 (pitch start). Hard stop 16:00.

Friday 5 June is a 7-hour working day (09:00–16:00) rather than the standard 8-hour day.

Teams may rehearse internally as they see fit. This is a deliberate choice to preserve assessment authenticity at the cost of marginally higher performance risk on pitch day.

9. Deliverables

9.1 Team-level Deliverables (per team)

#	Deliverable	Form	Due
T1	GitHub fork of iDaVIE with full assessment history.	Repository	Continuous; frozen Thu 4 June 11:00.
T2	Baseline maintainability benchmark report.	PDF + raw tool exports	Day 2 (Tue 19 May).
T3	Architecture overview document (C4 + ADR log + plug-in ABI specification).	PDF	Day 10 (Fri 29 May).
T4	Consolidated refactoring proposal report (max 60 pages excluding appendices).	PDF	Frozen Thu 4 June 11:00; submitted Fri 5 June 14:00–16:00.
T5	Pitch: 40 min talk + 20 min Q&A to the iDaVIE panel.	Live + slide deck PDF	Thu 4 June 11:00–12:00 (Alpha) / 12:00–13:00 (Beta).
T6	CI/CD pipeline producing the metric dashboard on every push (owned collectively by Quality Guild).	GitHub Actions + dashboard URL	Operational by Day 3; final state Thu 4 June 11:00.
T7	Team Integration & Metrics Report (joint deliverable signed by all 7 Tech Leads).	PDF	Frozen Thu 4 June 11:00.
T8	AI-tool usage log and reflection (one team-level document, signed by all sub-teams).	PDF, max 8 pages	Frozen Thu 4 June 11:00.

9.2 Sub-team Deliverables (per sub-team)

Per the per-sub-team specifications in Section 6 plus the following common artefacts. For Team Beta's 3-person Persistence sub-team, the scope adjustments in Section 6.7.2 apply.

1. Sub-team requirements document (1–2 pages).
2. Sub-team design document (5–10 pages for 4-person sub-teams; 4–7 pages for Team Beta Persistence).
3. Worked refactoring examples — two per 4-person sub-team, one for Team Beta Persistence.
4. Sub-team test strategy (2–4 pages).
5. Sub-team Kanban/Trello snapshot at the end of each sprint.

6. Daily stand-up notes (single shared file).

9.3 Per-student Deliverables

- Two-page individual reflection covering personal sub-team contributions, two software-engineering lessons learned, and an explicit description of AI tool use, helps and misses. Due Fri 5 June by Brightspace deadline.
- Peer-rated contribution table submitted privately. Due Fri 5 June by Brightspace deadline.

10. Process, Roles, Tooling and AI Policy

10.1 Three-layer Scrum Design

This assessment uses a Scrum-of-Scrums adaptation with three layers of roles.

Layer 1: Team-level Roles (3 per team)

Role	Responsibility	How filled
Team Product Owner	Single voice to the iDaVIE panel. Owns team vision and architectural drivers, prioritises across sub-teams, writes team-level user stories.	Rotated each sprint. Team Alpha pool: 28 students. Team Beta pool: 24 students (six four-person sub-teams; Persistence excluded). Elevation availability does not influence grading.
Team Scrum Master	Team-wide process facilitating the cross-sub-team stand-up, runs team sprint reviews and retrospectives, owns the team-wide Kanban.	Rotated each sprint. Same eligibility pools as Team PO above.
Integration Lead	Cross-cutting technical decisions and end-to-end integration. Chairs Architecture Guild. Maintains dependency graph and integration risk register.	Stable across the assessment. Held by the Architecture sub-team's Tech Lead, who does not rotate the TL role within Architecture (carve-out from the standard rotation rule).

Team PO and Team SM elevations are full-time for the elevated sprint: the student is seconded out of their sub-team for those five days. The vacated sub-team operates at 3 members for that sprint. Across 3 sprints, this distributes across 6 different sub-team-sprint cells (never the same sub-team twice in a row).

Layer 2: Sub-team Roles

Four-person sub-teams rotate four roles each sprint:

Role	Responsibility	Rotation
Sub-team Scrum Master	Runs daily 09:00 stand-up (08:55 on interview days). Removes impediments inside the sub-team. Maintains sub-team Kanban column.	Each sprint
Sub-team Tech Lead	Technical decisions within the sub-team. Attends 09:15 cross-sub-team stand-up. Sits on Architecture Guild.	Each sprint (except Architecture sub-team — see Integration Lead).

Sub-team PO Liaison	Translates Team PO priorities into sub-team backlog. Owns the sub-team's slice of the team backlog.	Each sprint
Sub-team Quality Champion	Metrics, CI status, baseline benchmark, worked-refactoring evidence for the sub-team. Sits on Quality Guild.	Each sprint

Standard rotation across three sprints — every student holds three of the four roles:

Sprint	Student A	Student B	Student C	Student D
Sprint 1	Scrum Master	Tech Lead	PO Liaison	Quality Champion
Sprint 2	Quality Champion	Scrum Master	Tech Lead	PO Liaison
Sprint 3	PO Liaison	Quality Champion	Scrum Master	Tech Lead

Team Beta's 3-person Persistence sub-team uses a different pattern. With three students and four roles, one student per sprint bundles SM+QC. Each student does the bundle once and each single role once across three sprints:

Sprint	Student X	Student Y	Student Z
Sprint 1	Scrum Master + Quality Champion	Tech Lead	PO Liaison
Sprint 2	PO Liaison	Scrum Master + Quality Champion	Tech Lead
Sprint 3	Tech Lead	PO Liaison	Scrum Master + Quality Champion

The SM+QC bundle pairs the two coordination-heavy roles (process + metrics). If any member of Team Beta Persistence is absent for more than half a day, the fall-back applies the 2-person rules — SM+QC paired with one student; TL+POL paired with the other.

Layer 3: Guilds (cross-cutting)

No single sub-team coordinates quality, CI/CD, metrics or benchmarking. Two standing guilds operate horizontally:

Guild	Membership	Facilitator	Cadence
Architecture Guild	7 sub-team Tech Leads	Integration Lead (stable; from Architecture sub-team)	Daily at 09:15 (08:55 on interview days). Weekly 1-h review on Wednesdays.

Quality Guild	7 sub-team Quality Champions. Team Beta facilitator pool excludes the Persistence QC (bundled with SM).	Rotating each sprint among the 6 four-person sub-teams' Quality Champions.	Daily 15-min huddle 10:30 (post morning break). Weekly metrics review 1 h Fridays before sprint review.
----------------------	---	--	---

The Quality Guild collectively owns the team's CI/CD pipeline, metrics dashboard and team-level Integration & Metrics Report. No single sub-team owns these, but they get done because the Guild is accountable as a group.

10.2 Scrum Ceremonies

Ceremony	Level	When	Duration
Sub-team daily stand-up	Sub-team	Daily 09:00 (async on interview days)	10 min
Cross-sub-team stand-up	Team	Daily 09:15 (08:55 on interview days)	15 min
Quality Guild huddle	Cross-cutting	Daily 10:30	15 min
Sprint planning	Both	Day 2 (Sprint 1), Day 6 (Sprint 2). No formal Sprint 3 planning.	2h sub-team + 1h team
Sprint review	Team	Day 5, Day 10. Final review = pitch on Day 14.	1h
Sprint retrospective	Sub-team + Team	Day 5, Day 10. Sprint 3 retro: Tue 2 June 16:00–17:00 in sub-teams only.	1h sub-team + 30 min team
Architecture Guild review	Cross-cutting	Wednesdays	1h

10.3 CI/CD

- A working CI pipeline by end of Day 1 (compiles + runs static analysis on one file).
- By end of Sprint 1, the pipeline runs the full metric suite on every PR and posts the dashboard delta as a PR comment.
- Quality gates harden across the assessment: by Day 10 a PR introducing an architecture violation, a circular dependency or a CK threshold breach is blocked from merge.
- All worked refactoring example code lives in /refactoring-examples (one folder per sub-team).

10.4 Modelling notation

- UML default for class, component, sequence and state-machine diagrams.
- SysML for requirements diagrams, block definition diagrams (BDD) and parametric diagrams where appropriate.
- All diagrams source-controlled in a text-based format (PlantUML, Mermaid, .drawio XML); no binary-only diagrams.

10.5 AI / GenAI usage policy

AI tools are not just permitted, they are expected. The assessment evaluates how well students use AI, not whether they avoid it.

1. Any AI-generated artefact must be reviewed, understood and defensible by the submitting student or sub-team.
2. Every sub-team maintains an AI tool usage log. i.e. tool, model, prompt class, where it helped, where it failed, what the human did instead. Consolidated at team level in deliverable T8.
3. Verbatim AI output must not be passed off as human-authored prose in the final report.
4. During the pitch and interviews, the iDaVIE panel may ask any team member to explain any design decision or any piece of code in their report. Inability to explain, regardless of whether AI was used, is a fail signal for that section.
5. AI tools MAY BE used for requirements drafting, ADR drafting, code skeletons, test generation, refactoring proposals, diagram generation, metric interpretation and prose editing etc.
6. AI tools MAY NOT be used for peer-rating, contribution log, individual reflection, or live pitch/interview defence.

Appendix A: ISO/IEC 25010 Maintainability

Sub-characteristic	Definition (ISO/IEC 25010:2023)	Primary metric families
Modularity	Degree to which a system is composed of discrete components such that a change to one has minimal impact on others.	Coupling, cohesion, dependency cycles, fan-in/fan-out, instability, package dependencies, architecture violations.
Reusability	Degree to which a product can be used as an asset in more than one system or in building other assets.	Interface stability, dependency isolation, package boundary clarity, ABI versioning.
Analysability	Effectiveness and efficiency of assessing change impact, diagnosing deficiencies or identifying parts to modify.	Cyclomatic complexity, cognitive complexity, LOC, method/class size, nesting depth, duplication, code smells, documentation density.
Modifiability	Degree to which a product can be effectively and efficiently modified without introducing defects or degrading quality.	Change impact, coupling, duplication, instability, propagation cost, technical debt, maintainability index, churn, hotspots.
Testability	Effectiveness and efficiency of establishing test criteria and performing tests to determine whether they are met.	Cyclomatic complexity, branch/line coverage, condition coverage, dependency isolation, mocking difficulty, interface size, fan-out, test-affecting code smells.

Appendix B: Tool Matrix

Tool	Primary outputs	Owner	Cadence
SonarQube Cloud	Code smells, complexity, duplication, maintainability rating, technical debt, coverage.	Quality Guild	Every PR + nightly.
Understand	CK metrics, structural metrics, dependency browsing.	Quality Guild	Sprint boundary snapshots.
NDepend (Depend)	CQLinq architecture rules, instability, propagation cost, layer enforcement.	Architecture Guild (rules) + Quality Guild (CI)	Every PR.
CodeScene	Hotspots, churn, knowledge map, code health, change coupling.	Quality Guild	Daily; full report Day 2 and Day 12.
DV8	Dependency Structure Matrix, propagation cost, architectural anti-patterns.	Architecture Guild + Quality Guild	End of each sprint.
GitHub Actions	CI/CD orchestration; PR comments; dashboard publishing.	Quality Guild	Continuous.

Appendix C: Pitch Format (Thu 4 June, 2026)

Each team is allocated a 60-minute pitch slot:

1. 40 min presentation. Suggested structure: 4 min iDaVIE pain points with metrics; 10 min target architecture (C4 levels); 12 min worked refactoring examples with before/after CK numbers; 6 min testability and Unity 6 migration plan; 5 min trade-offs and risk; 3 min summary.
2. 20 min Q&A from the iDaVIE maintainer panel. Panel may direct questions at any named team member. The Tech Leads and the Architecture sub-team must be present.
3. Slides presented from PDF; recording made for moderation. Pitch held in a lecture theatre, not in either team's workspace.

Team Alpha: 11:00–12:00. Team Beta: 12:00–13:00 (documented lunch exception; lunch shifts to 13:00–14:00 on this day only). No written panel feedback is provided; grades are returned via Brightspace after moderation.